ActiMM: Bridging Skip and Efficiency in GPU Sparse Convolution for 3D Point Cloud Processing

Anonymous Author(s)

Abstract

Sparse convolution is the dominant primitive for 3D point cloud perception, yet existing GPU implementations face a fundamental tradeoff between *computation skipping* and *kernel efficiency*. Gather-GEMM-scatter approaches (e.g., MinkowskiEngine) skip empty neighbors entirely but suffer from per-offset kernel launch overhead and load imbalance. Implicit GEMM approaches (e.g., SpConv v2, TorchSparse++) fuse all offsets into a single efficient kernel but cannot skip empty neighbors, incurring redundant computation proportional to sparsity.

We present ActiMM, a sparse convolution runtime that eliminates this tradeoff through two complementary execution models. *Skip GEMM* precomputes all active neighbor pairs and executes them as per-offset dense GEMM tiles within a single persistent-thread kernel, achieving computation strictly proportional to active pairs. *Elastic GEMM* dynamically redistributes idle threads within each output-stationary thread block to cooperatively process different channel segments of active points, trading a lightweight reduce for multiple eliminated K-loop iterations.

On an RTX 2080 Ti with FP32 precision, ActiMM's Skip GEMM is faster than all existing sparse convolution systems across the entire active-ratio spectrum: 11.5× faster than the best baseline at 1% active ratio, and still 1.9× faster at 30%.

1 Introduction

Sparse convolution is the computational backbone of 3D point cloud perception. Applications ranging from LiDAR-based autonomous driving [5, 6] to 3D semantic segmentation [1] and scene reconstruction rely on sparse convolution to process the inherently sparse voxelized representations of 3D sensor data. Unlike dense 2D convolution, where every spatial location participates in computation, 3D point clouds exhibit extreme sparsity: typical LiDAR scans occupy fewer than 5% of the voxel grid, and early network layers may see active ratios below 0.5%.

This sparsity creates a fundamental tension in GPU kernel design. On one hand, *skipping empty neighbors* avoids wasted computation—a voxel with only 3 active neighbors out of 27 in a 3×3×3 kernel should ideally perform 9× less work than a dense convolution. On the other hand, GPU efficiency demands *regular, coalesced memory access and high arithmetic intensity*—properties that are difficult to maintain when the computation pattern is irregular and data-dependent.

Existing sparse convolution libraries resolve this tension by committing to one side. The *gather-GEMM-scatter* approach (MinkowskiEngine [2]) iterates over kernel offsets, gathers active input features into a dense buffer, performs a standard GEMM, and scatters results back. This achieves perfect computation skipping—only active neighbor pairs are computed—but pays a heavy price: up to 27 separate kernel launches per layer, severe load imbalance across offsets, and redundant memory traffic from explicit gather/scatter buffers. The *implicit GEMM* approach (SpConv v2 [3], TorchSparse++ [4]) fuses all offsets into a single output-stationary GEMM kernel, achieving high GPU utilization and Tensor Core compatibility. However, implicit GEMM tiles the full $K = K_{\text{vol}} \times C_{\text{in}}$ dimension, including offsets where a given output point has no active neighbor. These empty slots produce zero-valued input features that still consume memory bandwidth and MAC cycles. TorchSparse++ mitigates this through bitmask sorting—grouping output points with similar active-neighbor patterns so that entire K-tiles can be skipped when no point in a warp has an active neighbor at that offset—but the skip granularity is coarse (warp × K-tile), and residual redundancy of 1.2–2.9× remains.

Table 1. Kernel latency (ms) for SubM 3×3×3 convolution, C_{in}=C_{out}=128, spatial 128×256×16, FP32, RTX 2080 Ti. ActiMM's Skip GEMM is faster than all existing systems across the entire active-ratio spectrum. Speedup is relative to the fastest baseline at each ratio.

Active ratio	ActiMM Skip GEMM	SpConv v2	TorchSparse++	speedup
0.1%	**0.044**	0.385	0.613	8.8×
0.5%	**0.058**	0.555	0.776	9.6×
1%	**0.056**	0.645	1.116	11.5×
5%	**0.300**	1.165	2.662	3.9×
10%	**0.961**	2.863	5.226	3.0×
20%	**4.218**	8.792	10.459	2.1×
30%	**8.094**	15.442	16.013	1.9×

We present ActiMM, a sparse convolution runtime that bridges this gap through two complementary execution models.

Skip GEMM: near-optimal at high sparsity. Rather than iterating over offsets (gather-GEMM-scatter) or tiling the full $K_{\text{vol}} \times C_{\text{in}}$ dimension (implicit GEMM), Skip GEMM *precomputes all active (input, output) neighbor pairs*, groups them by kernel offset, and executes them as per-offset dense GEMM tiles within a single persistent-thread kernel. Empty offsets produce zero pairs and are skipped entirely—no wasted

computation, no redundant memory access. Unlike gather-GEMM-scatter, all offsets are fused into one kernel launch with automatic load balancing via the persistent-thread stride loop.

Table 1 shows the result: in FP32 on an RTX 2080 Ti, ActiMM's Skip GEMM is faster than all existing sparse convolution systems *across the entire active-ratio spectrum.* At 1% active ratio (typical for early layers of LiDAR detection networks), Skip GEMM is **11.5**× faster than the best baseline (SpConv v2). Even at 30% active ratio, Skip GEMM maintains a **1.9**× advantage.

This advantage reflects a fundamental difference in execution model. Implicit GEMM is output-stationary: each thread block owns a fixed set of output points and iterates over all kernel offsets, including empty ones. Skip GEMM is weight-stationary: each tile owns a fixed kernel offset and iterates over only the active pairs for that offset. At high sparsity, the weight-stationary model processes work proportional to the number of active neighbor pairs ($\sum_\delta |\mathcal{M}_\delta|$), while the output-stationary model processes work proportional to the full kernel volume ($N_{\text{out}} \times K_{\text{vol}} \times C_{\text{in}}$). At 1% active ratio, Skip GEMM performs roughly 100× less useful work—this is not an implementation gap but a model gap.

At high sparsity, Skip GEMM is near-optimal: every MAC is useful, load balancing is automatic, and the only overhead is the worklist construction itself (a single GPU kernel, amortizable across submanifold convolution layers where the active set is preserved).

Elastic GEMM: closing the gap at moderate sparsity. As the active ratio rises, a structural bottleneck emerges in Skip GEMM. Because it is weight-stationary, different offset tiles write partial results to the same output rows via `atomicAdd`. At 30% active ratio, each output point receives contributions from ∼8 offsets on average, and the atomic writeback traffic grows proportionally. While Skip GEMM still wins in FP32 (Table 1), the margin narrows—and under Tensor Core precision (FP16/TF32), where TorchSparse++'s implicit GEMM gains a hardware compute advantage, the crossover shifts to ∼20–25%. In real detection networks such as CenterPoint [6], the deepest sparse convolution stage (after 3× stride-2 downsampling) operates at 35–57% active ratio with C_{in}=64–128—precisely the regime where Skip GEMM's writeback overhead is most acute.

To close this gap, we propose Elastic GEMM, an adaptive output-stationary kernel that eliminates the redundant computation of standard implicit GEMM without introducing atomic scatter. The key observation is that within an output-stationary thread block processing a 3×3×3 kernel, the number of active neighbors varies dramatically per offset: some offsets (e.g., the center) are active for nearly all output points, while peripheral offsets may be active for only a fraction. In standard implicit GEMM, threads assigned to inactive output points at a given offset compute zero-valued MACs—wasted cycles that cannot be reclaimed. Elastic GEMM *redistributes* these idle threads: when only $1/k$ of the output points in a tile have active neighbors at a given offset, all threads cooperatively process different input channel segments of the active points, reducing the K-loop iteration count for that offset by a factor of k. After the shortened K-loop, a lightweight shared-memory reduce combines the partial channel sums—trading one reduce (a shared-memory write + read + barrier) for $k-1$ eliminated K-tile iterations (each involving global loads + full GEMM compute). This trade is overwhelmingly favorable: at C_{in}=64 and BK=8, a 4× elastic reduction saves 6 K-tile iterations and costs one shared-memory round-trip.

Unified runtime. Together, Skip GEMM and Elastic GEMM form a unified runtime that covers the full sparsity spectrum: Skip GEMM handles the high-sparsity layers where per-pair skip dominates (0.1–10% active ratio), while Elastic GEMM handles the moderate-sparsity layers where output-stationary execution with adaptive compute redistribution is more efficient (20–50%+). The dispatch decision is made once per layer based on the active ratio, which is known after voxelization and preserved across submanifold convolution layers.

The contributions of this paper are:

1. **Skip GEMM**, a weight-stationary execution model for sparse convolution that achieves computation proportional to active neighbor pairs within a single persistent-thread kernel. At 1% active ratio, this is 11.5× faster than the best existing system in FP32.
2. **Elastic GEMM**, an adaptive output-stationary kernel that dynamically redistributes compute based on per-offset sparsity within each thread block, eliminating redundant computation without atomic scatter.
3. A **unified runtime** that selects between Skip GEMM and Elastic GEMM per layer, targeting end-to-end speedup across the full sparsity spectrum of real 3D perception models.

References

[1] Jens Behley, Martin Garbade, Andres Milioto, Jan Quenzel, Sven Behnke, Cyrill Stachniss, and Jurgen Gall. 2019. SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences. In *ICCV*.

[2] Christopher Choy, JunYoung Gwak, and Silvio Savarese. 2019. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *CVPR*.

[3] Lue Fan, Feng Wang, Ziqi Pang, Lei Zhu, Zhicheng Su, Fang Chen, Naiyan Wang, and Zhaoxiang Zhang. 2022. Embracing Single Stride 3D Object Detector with Sparse Transformer. In *CVPR*.

[4] Haotian Tang, Shang Yang, Zhijian Liu, Ke Hong, Zhongming Yu, Xiuyu Li, Guohao Dai, Yu Wang, and Song Han. 2023. TorchSparse++: Efficient Training and Inference Framework for Sparse Convolution on GPUs. In *MICRO*.

[5] Yan Yan, Yuxing Mao, and Bo Li. 2018. SECOND: Sparsely Embedded Convolutional Detection. *Sensors*.

[6] Tianwei Yin, Xingyi Zhou, and Philipp Krähenbühl. 2021. Center-based 3D Object Detection and Tracking. In *CVPR*.